

UNIVERSITY OF ST ANDREWS

CS4302 PRACTICAL 3

Image Processing

Author:
150008022

November 11, 2018



Goal

The goal of this practical was to learn and apply image processing techniques using Matlab[®].

Tasks

The scripts are separated by task given in the practical specification. Where useful, scripts were created to hold functions that were used repeatedly. To run each practical task in order, the script *practical.m* can be used (This will generate a large number of figures) The images for processing are shown in figures 1, 2, and 3.



Figure 1: Image with large variation in colour and spatial frequency.

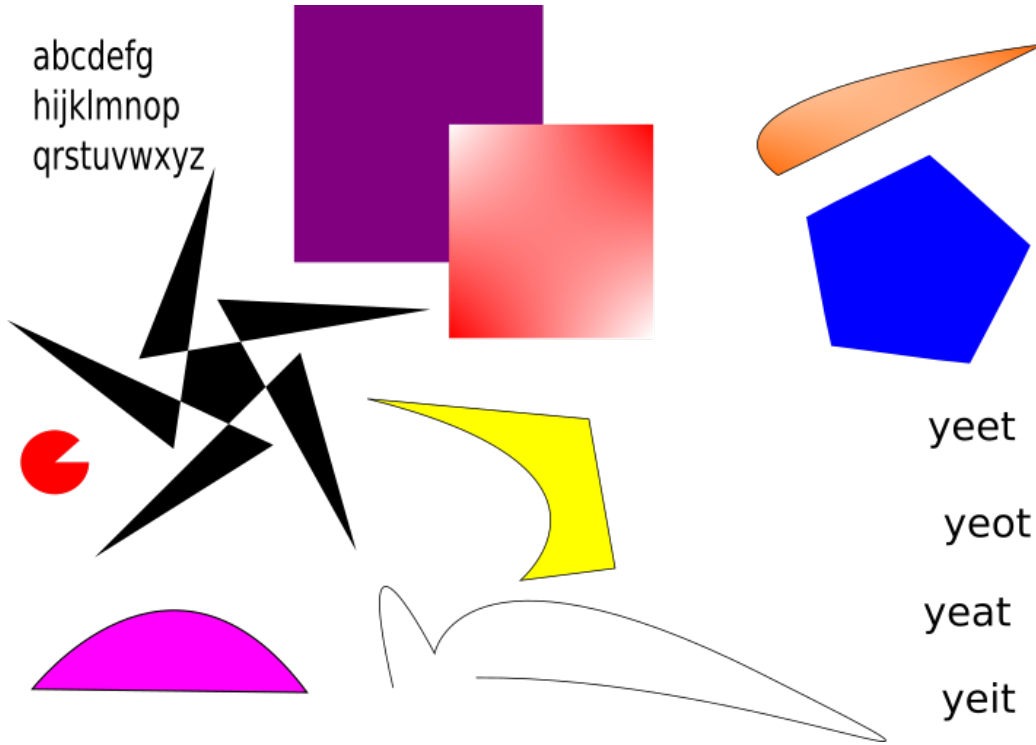


Figure 2: Vector image featuring text, polygons, and bezier curves. Originally created in Inkscape and exported as a PNG file.

1 Middle C

Middle C is approximately 261.6Hz, though concert pitch has varied historically. To plot this, a sine wave of the same frequency was generated.

Originally a sampling frequency of $2f$ was used, as Nyquist theorem dictates that the sampling rate must be at least double the highest frequency in the signal. However, MATLAB refused to play at this rate, as the documentation states that only values of $1000Hz \leq Fs \leq 384000Hz$ are accepted by the audioplayer function [1]. Instead, the sample rate used for high quality music was used instead, $48kHz$. Graphing the wave at $1000Hz$ showed a far less smooth wave in the time domain, though it did not affect the frequency domain plot (Figure .

To generate ten seconds of audio, the length of a single sample was de-



Figure 3: Simple image with little variation in colour or spatial frequency.

terminated using the equation

$$P = \frac{1}{f}$$

where P is the duration in seconds and f is the sample frequency in hertz. $\frac{10}{P}$ then produced the number of samples necessary to produce ten seconds of audio.

In order to produce the amplitude over time for the given frequency, we first needed to convert the frequency from hertz to radians per second using the equation

$$\omega = 2\pi f$$

Then we could use the simple harmonic oscillator equation with $\phi = 0$ and $A = 1$ to produce the displacement from equilibrium of the wave at some time t

$$y = A \sin(\omega t - \phi)$$

Matlab greatly simplified the process of producing these values over a range of time through its use of matrices as a standard data type. Out of curiosity, the value of A was changed to see if the resulting sound was louder when played later, and this was confirmed to be true as expected.

The time domain was then graphed, and zoomed in by a factor of 100 so that the actual wave structure was visible. The unit of metres was used for the amplitude, though the perceived amplitude of the sound would actually be dependant on speakers and external settings when played.

Since plotting the frequency domain was such a common task in this practical, it was abstracted into a function script *frequencyDomainPlot.m*. First, the Nyquist limit was calculated using the sample frequency so that frequencies that could not exist within the signal were not plotted. Fast Fourier Transform was then applied to the signal, and shifted so that the zero frequency component was centred in the array. The magnitude axis was produced by taking the second half of values in the fft matrix and taking their absolute value. The magnitude values for negative frequencies, which were not useful for plotting, though they were necessary for inverting the fft later on. The frequency axis was generated by producing evenly spaced values from 0 to Nyquist limit. This produced the bottom graph in Figure ???. Zooming into the graph was enabled using *zoom on*, and this showed that the peak appeared at 261Hz as hoped. Experimenting with various durations of signal also showed that this affected the accuracy of the fft, spreading the peak over a larger range of frequencies as the duration decreased (Figure The change in time domain plot is only due to the constant zoom being applied to a short range of x-axis values, not an actual change of frequency.

The sound produced had a constant intensity and absolutely no variation in pitch. The lack of any other frequencies results in a very unnatural sound.

2 Time Domain

Plotting the time domain followed a similar process as in Task 1. Instead of hard coding a sample frequency however, it was supplied by the audio file itself. From then on, the same process could be applied to the signal as before to produce Figure ?? and Figure Both plots show that the signals are roughly equally as dense. There also appears to be more noise present

in the voice recording, as the 'silence' between blocks of speech appears far rougher than the silence between notes in the music plot. There is also more variation in amplitude in the voice plot.

3 Frequency Domain

The same function used for Task 1 was reused here, but this time it produced a more interesting spectrum of frequencies. The high sample used in both files resulted in a large empty space in the original frequency plot, and so a limit was applied so that only frequencies up until $7kHz$ were plotted in Figure

The voice plot appears to have a much more diverse spectrum compared to the music plot. This is likely due to the music only containing frequencies belonging to the notes being played, which are usually constrained within a key. The glissando near the end of the music likely introduced more frequencies outside of the key however.

The most prevalent frequency in the voice plot is around $125Hz$, which lies on the upper end of the range of male voice fundamental frequencies. The most prevalent frequency in the music plot is around $386Hz$, which is likely to be a 4th octave G in an equal tempered scale [3].

4 Frequency Filtering

For frequency filtering, I used two methods. The first is my own implementation and the other is using Matlab's *bandpass* function.

For my custom bandpass filter, I took the following steps:

1. Produce the fft of the signal and perform the fft shift.
2. Produce a frequency axis for the signal.
3. Zero any values in the frequency axis that are outside the frequency band specified, both in the positive and negative frequencies.
4. Set all other values in the frequency axis to 1.
5. Multiply the fft matrix by the frequency matrix, effectively zeroing any coefficients that belong to unwanted frequencies.

6. Reverse the fftshift and fft to produce the original signal.

This allowed the same function to be used as both a low-pass (0 to x) and high-pass (x to ∞) filter, as well as a bandpass filter.

I applied the same filter to both, allowing frequencies in the range 1kHz to 2.5kHz. This removed the most prevalent frequencies from both sound files, whilst still leaving audible features. Both recordings gained a sinister feel. Words became unintelligible in the voice recording. Despite knowing exactly what was said, it was incredibly difficult to work out which part of the clip corresponded to which word, even though the time domain plot still retained most of the original features. The loudness of the track was drastically reduced.

The music recording still held most of its features, especially the glissando towards the end. The track sounded as though it was being played through an old radio. After realising the filter had a similar effect on sound as the tone control on an electric guitar, I found out that this control reduces higher frequencies in the output, which is what had also occurred with the signal in the custom filter.

The Matlab bandpass filter was less destructive. For the voice filter, words were still unintelligible, and it sounded as though I had a cold. Looking into this, I found that our nose and sinuses act as a resonance chamber, which essentially amplifies certain frequencies in speech, mostly the lower frequencies. Similar to the effect the hollow body of a guitar has on the sound it produces.

5 Identifying Noise

In order to identify the noise, a frequency plot had to be produced. Using fast Fourier transform as before, Figure was produced which clearly shows the frequency of the incredibly unpleasant whine. Zooming in shows that the peak occurred at 500Hz.

6 Removing Noise

To remove the noise, two approaches were taken.

The first involved using a bandstop, which removed only the small frequency range that the noise occurred in. This unfortunately introduced a

metallic ringing sound to the blocks of speech in the audio, though it did a brilliant job of removing the high frequency whine (Figure . The high frequency ringing sound added to speech was further evidence that the Butterworth filter was used for Matlabs bandstop/bandpass functions, as one of the characteristics of this filter is overshoot and ringing in step response. The new peaks introduced in the frequency plot are likely the contributors to the ringing sound. The highest frequency was past the range of human hearing, and so its magnitude was not a problem.

The alternative approach involved using a low pass filter to remove everything below the noise frequency. This was also successful, though the ringing was still present above the voice. The large high frequency peaks also were not present, suggesting the ringing present in the voice was being caused by a lower frequency component.

Another method I attempted was to generate a signal of the same frequency as the noise and remove it by subtraction. However, this would require also knowing the phase of the signal, which would require a lot of brute forcing, or a more thorough analysis of the output of the fft.

7 Extension

Now that I had gained some confidence in my understanding of Matlab , I tried to implement time-domain scrambling as an extension. To do so, I first declared how large my frames should be in seconds. I then calculated how many samples per frame, and used this to break the signal into frames.

For the scrambling key, I created a numbered list from 1 to the number of frames, and then reordered them. The scrambled signal matrix was then constructed by appending the frame in the row number given by the scrambling key. Listening back, I can still identify parts of speech, but all meaning is lost. Using the *vertcat* function instead of iteratively appending the samples resulted in an incredibly large performance boost, which highlighted the effectiveness of Matlab for this sort of processing. The before and after effect of scrambling on the time domain plot can be seen in Figure ???. The same scrambling matrix could then be used to inverse the scrambling through a similar process, though the padding used to extend the signals to the same length for plotting would have to also be avoided as it would introduce silence to the middle of the audio.

Conclusion

This practical was incredibly enjoyable (excluding the whine), and greatly helped my understanding of Matlab. Actually implementing the technologies discussed in lectures resulted in many rabbit holes of learning, and a better understanding of the content. For example, the various types of audio filters and their pros and cons, and why it isn't as simple as 'blocking' the unwanted frequencies.

I realised only close to the deadline that since on the schools Windows installs, the older version of Matlab (2017a) is used, and it did not have the bandpass function yet. However, in 2018b, it does exist, and so the scripts will only work on the linux machines. Given more time, I would have liked to replace the bandpass function with the more specific filters available, such as elliptical and chebyshev filters.

References

- [1] Matlab documentation. <https://uk.mathworks.com/help/matlab/ref/audioplayer.html>. Accessed: 2018-11-2.
- [2] Kazumi Totaka. Mii channel music, 2006.
- [3] Frequencies for equal-tempered scale. <https://pages.mtu.edu/~suits/notefreqs.html>. Accessed: 2018-11-2.
- [4] Stephen Butterworth. On the theory of filter amplifiers. *Wireless Engineer*, 7:536–541, 1930.